

Decision Making and Looping Statements

if Statement

Syntax:

```
If condition:  
    statement
```

For Example:

```
age = 20
```

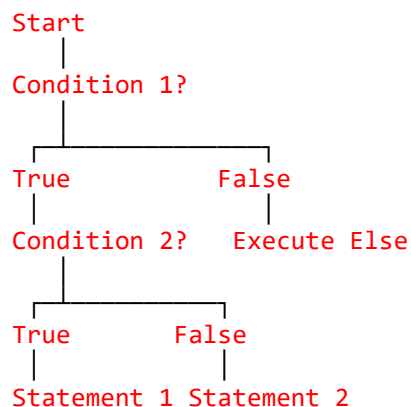
```
if age >= 18:  
    print("Eligible to vote")
```

Output:

```
Eligible to vote
```

Nested if-else statement

Flowchart



Syntax:

```
if condition1:  
    # Executes if condition1 is True  
  
    if condition2:  
        # Executes if both condition1 and condition2 are True  
    else:  
        # Executes if condition1 is True but condition2 is False  
  
else:  
    # Executes if condition1 is False
```

Example:

```
balance = 5000  
amount = 3000  
  
if balance >= amount:  
    if amount <= 10000:  
        print("Transaction Successful")
```

```
        else:
            print("Withdrawal Limit Exceeded")
    else:
        print("Insufficient Balance")
```

Output:

Transaction Successful

if-else statement

Syntax:

```
if condition:
    statements
else:
    statements
```

Example:

```
marks = 35
```

```
if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

Output:

Fail

if...elif...else

Syntax:

```
if condition:
    statements
elif condition:
    statements
elif condition:
    statements
else:
    statements
```

Example:

```
marks = 72
```

```
if marks > 100 or marks < 0:
    print("Invalid Input")
elif marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Output:

Grade C

switch case in Python (match-case)

Syntax:

```
match variable:
    case value1:
        statements
    case value2:
        statements
    case _:
        default
```

Example:

```
day = 3
```

```
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Invalid")
```

Output:

```
Wednesday
```

What is a Loop? (A loop repeats a block of code.)

for loop

Syntax:

```
for variable in sequence:  
    statements
```

for loop with range ()

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

for Loop with List

```
fruits = ["Apple", "Banana", "Mango"]
```

```
for fruit in fruits:  
    print(fruit)
```

Output:

```
Apple  
Banana  
Mango
```

range() function

Formats:

- `range(stop)`
- `range(start, stop)`
- `range(start, stop, step)`

Examples:

- `range(5)`

Output:

0 1 2 3 4

- `range(2, 8)`

Output:

2 3 4 5 6 7

- `range(2, 20, 2)`

Output:

2 4 6 8 10 12 14 16 18

Negative Step example:

- `range(5, 0, -1)`

Output

5 4 3 2 1

while loop

(Repeats while condition is True.)

Syntax:

```
while condition:
    statements
```

Example:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Output:

```
1
2
3
4
5
```

Infinite Loop

```
while True:
    print("Hello")
```

How to stop infinity loop?

- Ctrl + C
- or
- Use break

while Loop with else

- a while loop can have an optional else block.
- The else block executes only if the while loop finishes normally (i.e., the loop condition becomes False).
- If the loop is terminated using a break statement, the else block does not execute.

Syntax:

```
while condition:
    # Loop body
else:
    # Executes when the loop ends normally
```

Example 1:

```
count = 1

while count <= 5:
    print(count)
    count += 1
else:
    print("Loop completed successfully.")
```

Output

```
1
2
3
4
5
Loop completed successfully.
```

Example 2:

```
count = 1

while count <= 5:
    if count == 3:
        break

    print(count)
    count += 1
else:
    print("Loop completed successfully.")
```

Output

```
1
2
```

break, continue and pass

break Statement

(Stops the loop immediately)

Example:

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

continue Statement

(Skips current iteration)

```
for i in range(6):  
    if i == 3:  
        continue  
    print(i)
```

Output:

```
0  
1  
2  
4  
5
```

pass Statement

(Does nothing) Used as a placeholder

```
for i in range(5):  
    if i == 3:  
        pass  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

Statement	Purpose
break	Exit loop
continue	Skip current iteration
pass	Do nothing